

```

import pandas as pd
import numpy as np
import geopandas as gpd
import folium
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Step 1: Generate Synthetic Data
np.random.seed(42)
num_samples = 1000
data = {
    'latitude': np.random.uniform(low=34.0, high=34.1, size=num_samples),
    'longitude': np.random.uniform(low=-118.3, high=-118.2, size=num_samples),
}
data['deforestation'] = np.where(data['latitude'] + np.random.normal(0, 0.01, num_samples) > 34.05, 1, 0)
df = pd.DataFrame(data)
df.to_csv("deforestation_dataset.csv", index=False)
print("CSV file 'deforestation_dataset.csv' created successfully!")

# Step 2: Data Exploration
plt.figure(figsize=(10, 6))
plt.scatter(df['longitude'], df['latitude'], c=df['deforestation'], cmap='RdYlGn', alpha=0.5)
plt.title('Synthetic Data Distribution')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.colorbar(label='Deforestation (0 = No, 1 = Yes)')
plt.grid(True)
plt.show()

# Histograms
plt.figure(figsize=(8, 4))
sns.histplot(df['latitude'], bins=20, kde=True, color='skyblue')
plt.title("Histogram of Latitude")
plt.xlabel("Latitude")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 4))
sns.countplot(x='deforestation', data=df, palette='Set2')
plt.title("Bar Plot of Deforestation Classes")
plt.xlabel("Deforestation (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.grid(True)
plt.show()

sns.pairplot(df, hue="deforestation", palette='husl')
plt.suptitle("Pair Plot", y=1.02)
plt.show()

# Step 3: Prepare Data for Modeling
X = df[['latitude', 'longitude']]
y = df['deforestation']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Train Random Forest Model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Step 5: Make Predictions
predictions = model.predict(X_test)

# Step 6: Evaluate Model
print("\nModel Evaluation:")
print(classification_report(y_test, predictions))
print("Accuracy:", accuracy_score(y_test, predictions))

# Step 7: Create GeoDataFrame for Visualization
gdf_test = gpd.GeoDataFrame(X_test.copy(), geometry=gpd.points_from_xy(X_test.longitude, X_test.latitude))
gdf_test['predictions'] = predictions

# Step 8: Create Folium Map and Display Inline
m = folium.Map(location=[34.05, -118.25], zoom_start=12)

```

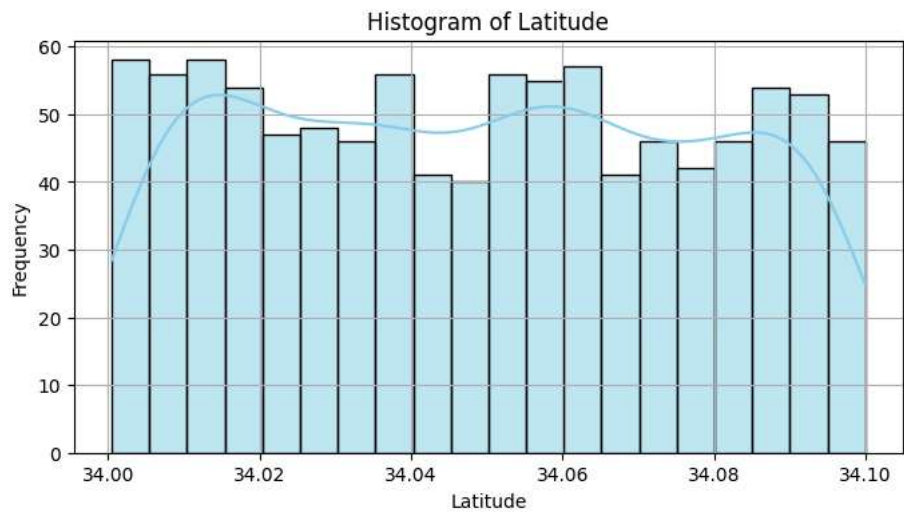
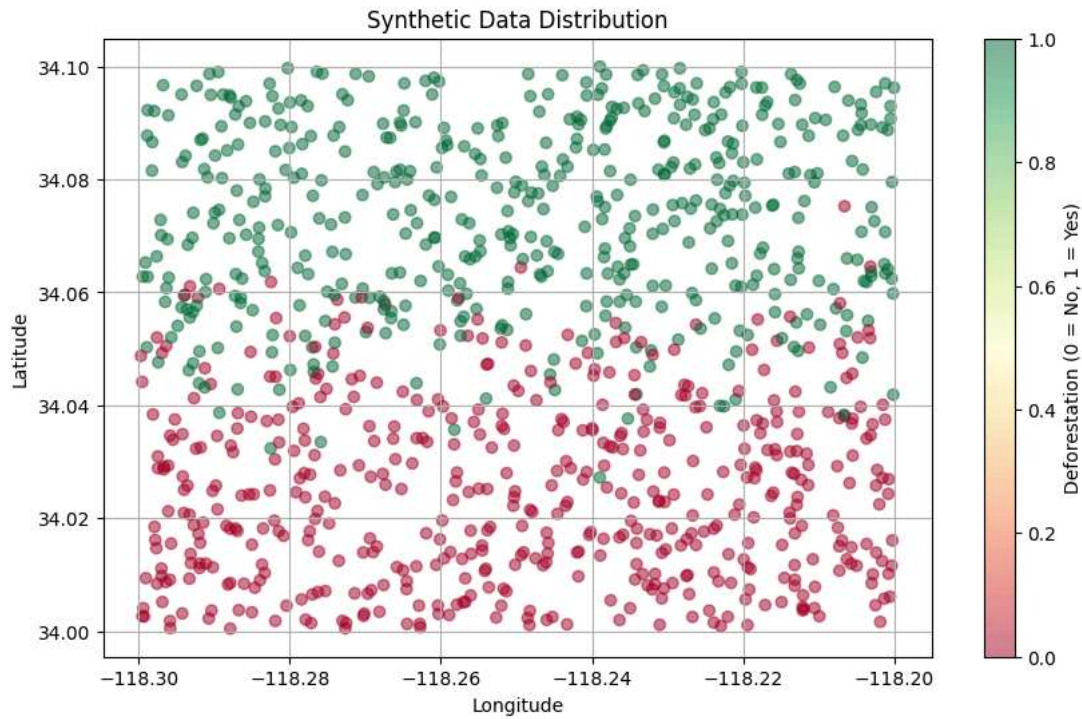
```
for idx, row in gdf_test.iterrows():
    color = 'green' if row['predictions'] == 0 else 'red'
    deforestation_status = "No Deforestation" if row['predictions'] == 0 else "Deforestation"
    folium.CircleMarker(
        location=(row['latitude'], row['longitude']),
        radius=5,
        color=color,
        fill=True,
        fill_opacity=0.6,
        popup=f'Deforestation Status: {deforestation_status}'
    ).add_to(m)

# Display Map Inline in Colab
m

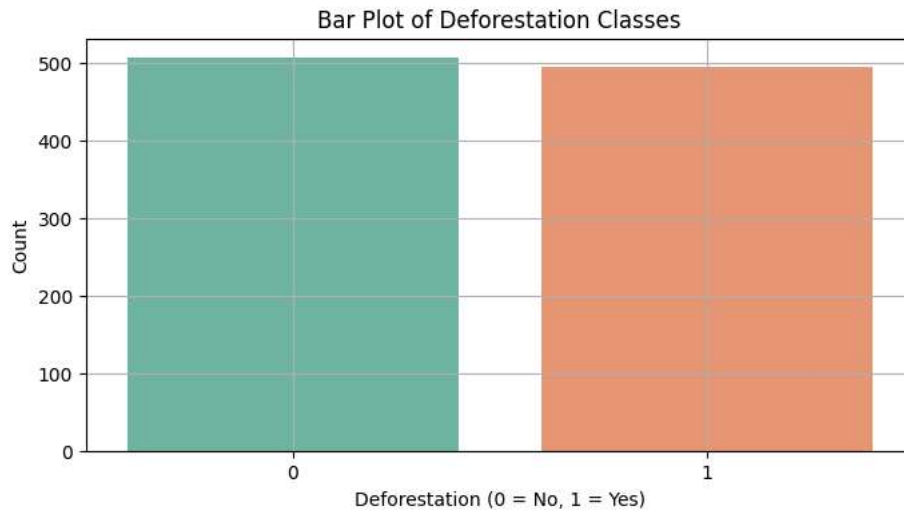
# Step 9: Save Map to HTML
m.save('deforestation_map.html')

# Step 10: Summary Report
print("\nSummary of Findings:")
print(f"Total Samples: {num_samples}")
print(f"Training Samples: {len(X_train)}")
print(f"Testing Samples: {len(X_test)}")
print(f"Model Accuracy: {accuracy_score(y_test, predictions):.2f}")
```

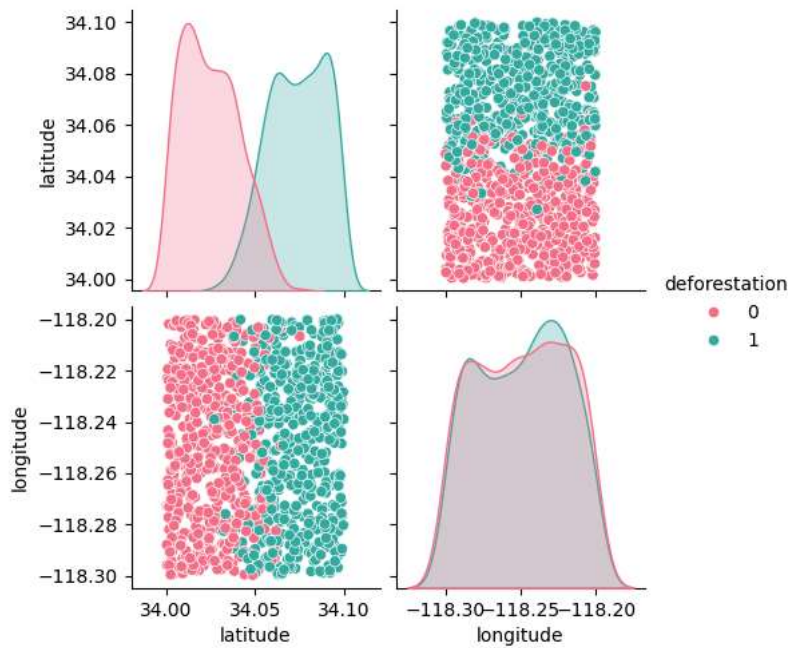
CSV file deforestation_dataset.csv created successfully:



```
<ipython-input-1-682df1d22c7b>:43: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend`
sns.countplot(x='deforestation', data=df, palette='Set2')
```



Pair Plot



Model Evaluation:

	precision	recall	f1-score	support
0	0.86	0.90	0.88	102
1	0.89	0.85	0.87	98
accuracy			0.88	200
macro avg	0.88	0.87	0.87	200
weighted avg	0.88	0.88	0.87	200

Accuracy: 0.875

Summary of Findings:
Total Samples: 1000
Training Samples: 800
Testing Samples: 200
Model Accuracy: 0.88

```

import pandas as pd
import numpy as np
import geopandas as gpd
import folium
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from IPython.display import display

# Step 1: Generate Synthetic Data for India
np.random.seed(42)
num_samples = 1000
data = {
    'latitude': np.random.uniform(low=8.0, high=37.0, size=num_samples),
    'longitude': np.random.uniform(low=68.0, high=97.0, size=num_samples),
}
data['deforestation'] = np.where(data['latitude'] + np.random.normal(0, 1, num_samples) > 22.5, 1, 0)
df = pd.DataFrame(data)
df.to_csv("deforestation_dataset_india.csv", index=False)
print("CSV file 'deforestation_dataset_india.csv' created successfully!")

# Step 2: Data Exploration
plt.figure(figsize=(10, 6))
plt.scatter(df['longitude'], df['latitude'], c=df['deforestation'], cmap='RdYlGn', alpha=0.5)
plt.title('Synthetic Data Distribution (India)')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.colorbar(label='Deforestation (0 = No, 1 = Yes)')
plt.grid(True)
plt.show()

# Histograms
plt.figure(figsize=(8, 4))
sns.histplot(df['latitude'], bins=20, kde=True, color='skyblue')
plt.title("Histogram of Latitude")
plt.xlabel("Latitude")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 4))
sns.countplot(x='deforestation', data=df, palette='Set2')
plt.title("Bar Plot of Deforestation Classes")
plt.xlabel("Deforestation (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.grid(True)
plt.show()

sns.pairplot(df, hue="deforestation", palette='husl')
plt.suptitle("Pair Plot", y=1.02)
plt.show()

# Step 3: Prepare Data for Modeling
X = df[['latitude', 'longitude']]
y = df['deforestation']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Train Random Forest Model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Step 5: Make Predictions
predictions = model.predict(X_test)

# Step 6: Evaluate Model
print("\nModel Evaluation:")
print(classification_report(y_test, predictions))
print("Accuracy:", accuracy_score(y_test, predictions))

# Step 7: Create GeoDataFrame for Visualization
gdf_test = gpd.GeoDataFrame(X_test.copy(), geometry=gpd.points_from_xy(X_test.longitude, X_test.latitude))
gdf_test['predictions'] = predictions

# Step 8: Create Folium Map (India Centered)

```